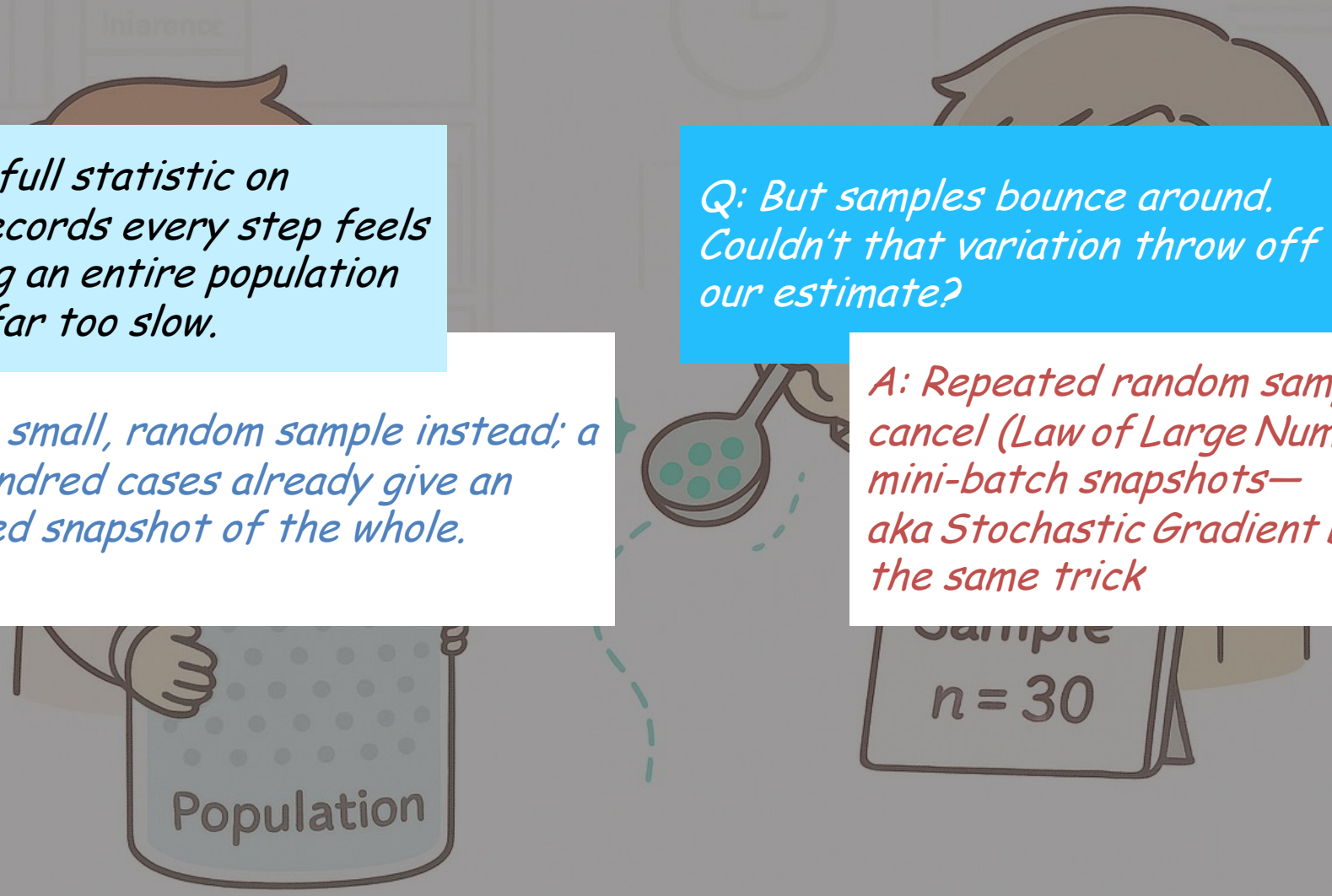
Machine Learning

Stochastic Gradient Descent

Tarapong Sreenuch

8 February 2024

克明峻德，格物致知



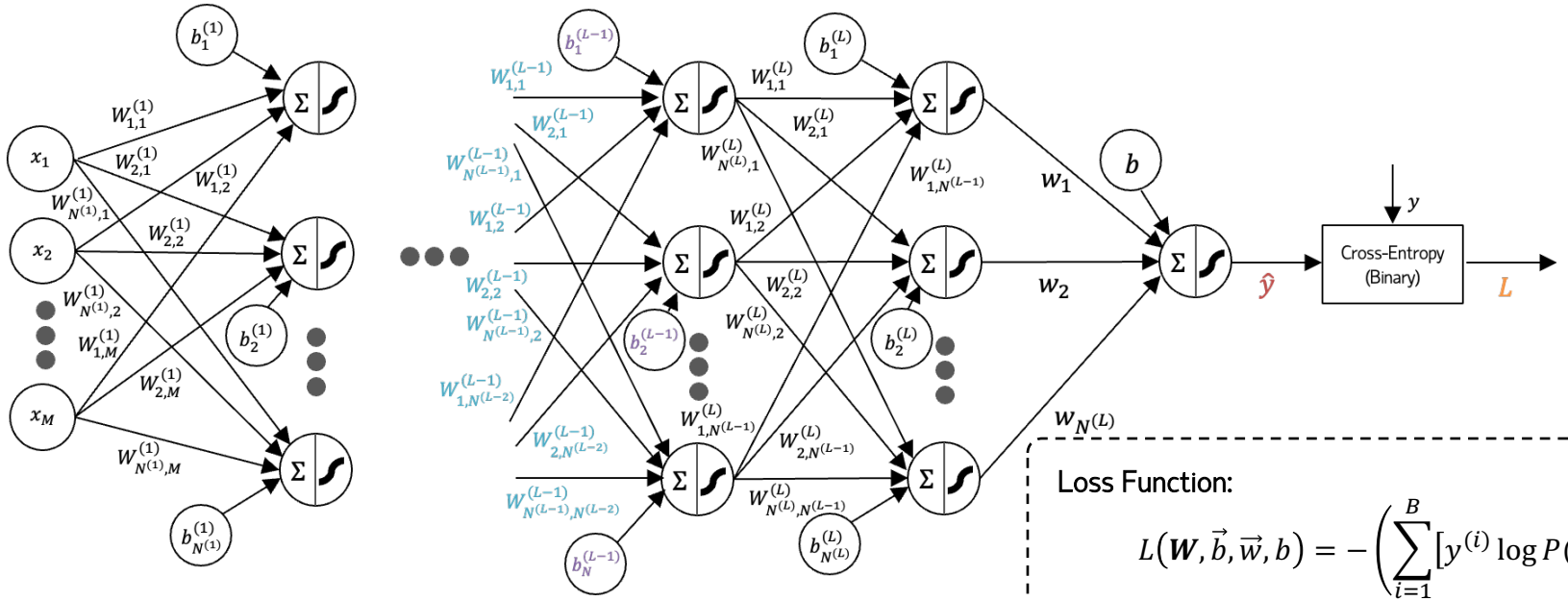
Computing a full statistic on millions of records every step feels like surveying an entire population each time - far too slow.

Draw a small, random sample instead; a few hundred cases already give an unbiased snapshot of the whole.

Q: But samples bounce around. Couldn't that variation throw off our estimate?

A: Repeated random samples let noise cancel (Law of Large Numbers); those mini-batch snapshots— aka Stochastic Gradient Descent—use the same trick

Deep Network



Forward Path:

$$\Phi^{(1)}(\vec{x}) = \sigma\left(\boldsymbol{W}^{(1)} \times \vec{x} + \vec{b}^{(1)}\right)$$

$$\vdots$$

$$\Phi^{(L-1)}(\vec{x}) = \sigma\left(\boldsymbol{W}^{(L-1)} \times \Phi^{(L-2)}(\vec{x}) + \vec{b}^{(L-1)}\right)$$

$$\Phi^{(L)}(\vec{x}) = \sigma\left(\boldsymbol{W}^{(L)} \times \Phi^{(L-2)}(\vec{x}) + \vec{b}^{(L)}\right)$$

$$\hat{y} = \sigma\left(\vec{w}^{T} \times \Phi^{(L)}(\vec{x}) + b\right)$$

Loss Function:

$$L(\boldsymbol{W}, \vec{b}, \vec{w}, b) = -\left(\sum_{i=1}^{B}\left[y^{(i)} \log P(1|\vec{x}^{(i)}, \vec{w}, b) + (1 - y^{(i)}) \log(1 - P(1|\vec{x}^{(i)}, \vec{w}, b))\right]\right)$$

Forward Pass: Each layer computes its output from the previous layer's numbers.

Backward Pass (Back-Prop): We now treat that loss as a function of every weight. Using the chain rule we calculate—layer by layer, in reverse order—the partial derivative of the loss with respect to each weight.

As models and datasets scale, every gradient-descent step must repeat heavy forward and backward passes, consuming massive CPU/GPU time and memory bandwidth—so training a deep network becomes a time-intensive process.

Q: Can we make it faster, but still the same result?

Why Sample? Estimating the Population with a Few Measurements

Population (N People)

Q: What are the heights of Singaporean males and females How can we determine this?

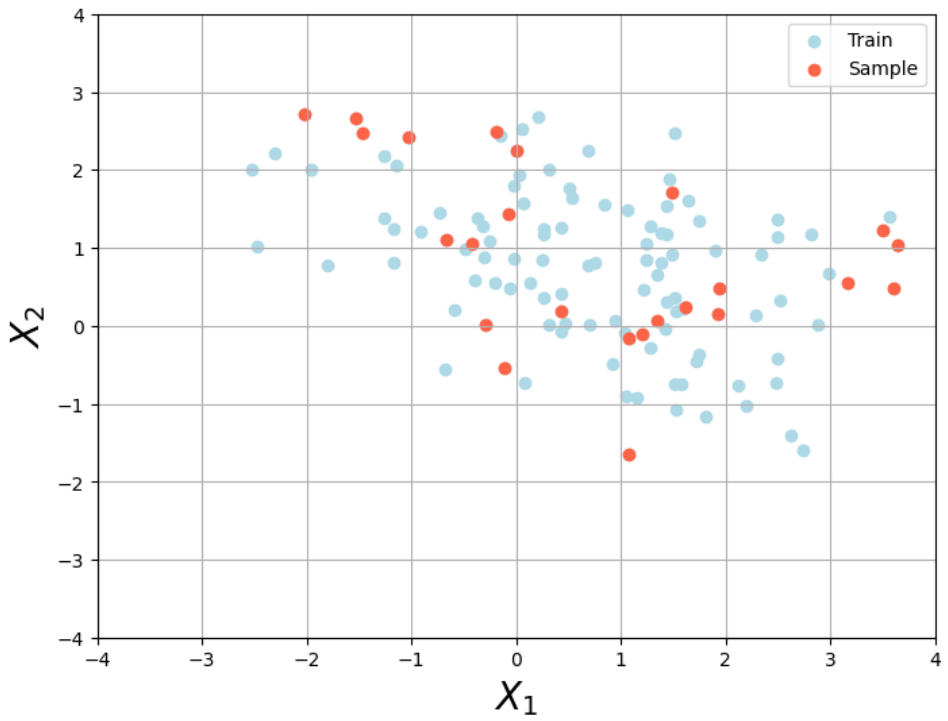
A: … Samples …

If our samples are representative, our estimates will be close to the the true values.

"Near enough is good enough."

Intuitively, estimates from samples approximate those from the full population.

Chain of Thoughts: SGD Intuition



Full-batch Gradient = Exact but Slow

Running one gradient step on the entire training set is precise, yet time-consuming when the data are large..

Single Sample? Too Narrow

Training on just one small subset is quick, but the model would never see most of the data, so the solution may be biased.

Mini-Batches = Fast Slices of the Whole

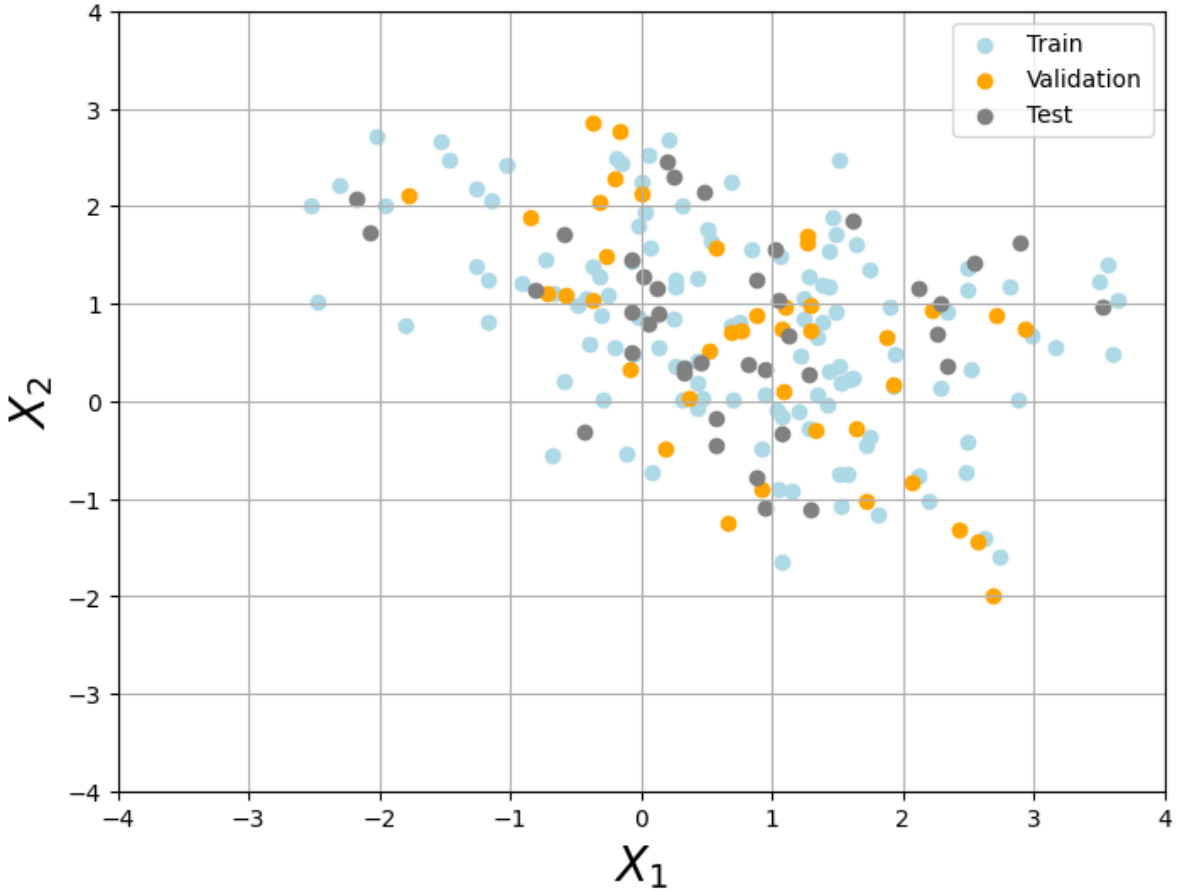
By drawing a fresh small batch each iteration, we keep each update cheap while ensuring that, over many steps, every training point is used.

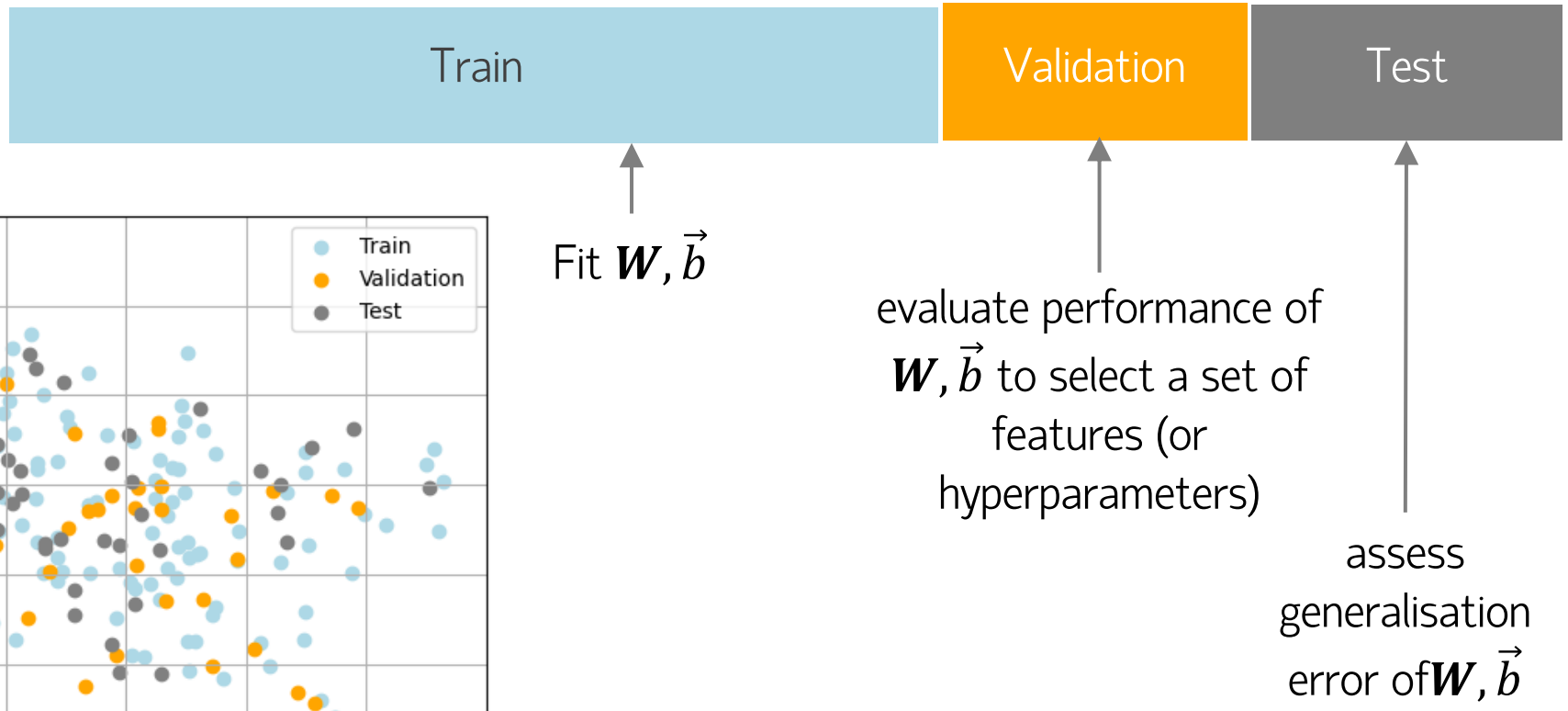
Same Destination, Less Time

As the mini-batches cycle through the dataset, their accumulated updates approximate the full-data gradient, giving nearly the same final model in far cheaper computationally.

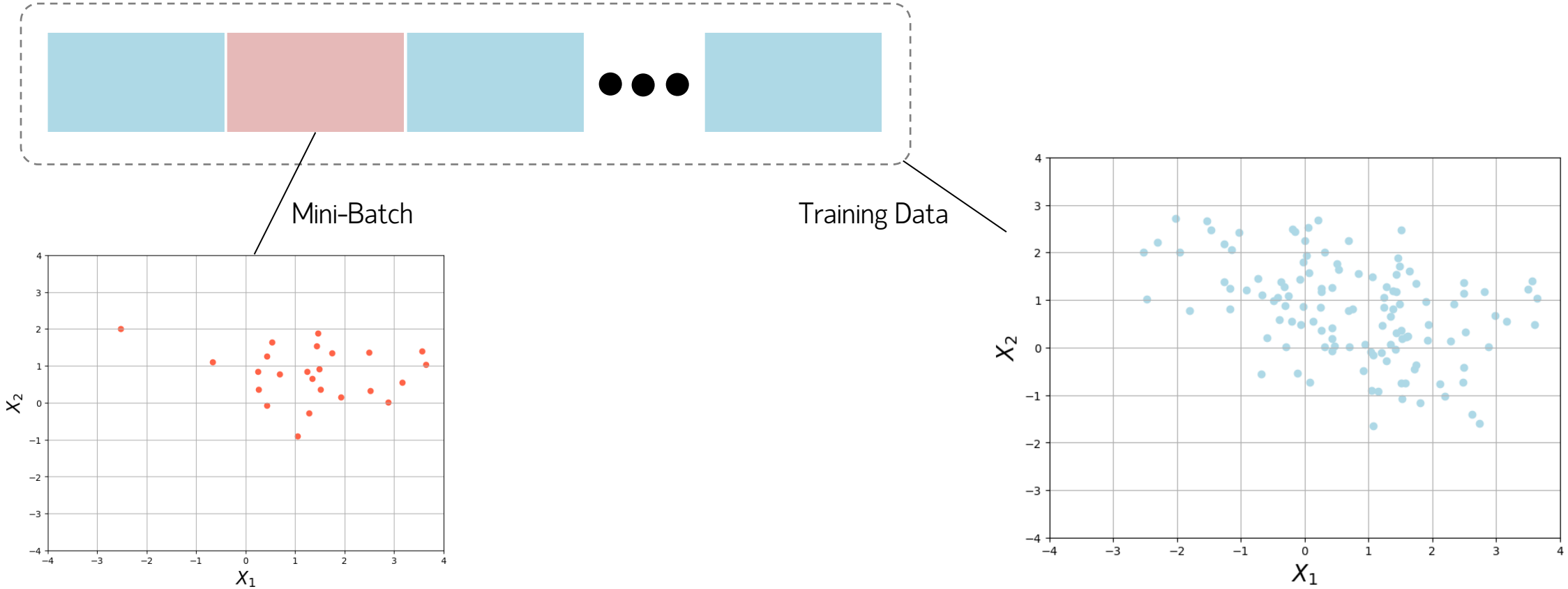
"Mini-batch SGD reaches the full-batch solution faster by trading one huge computation for many small, inexpensive ones."

Large Dataset



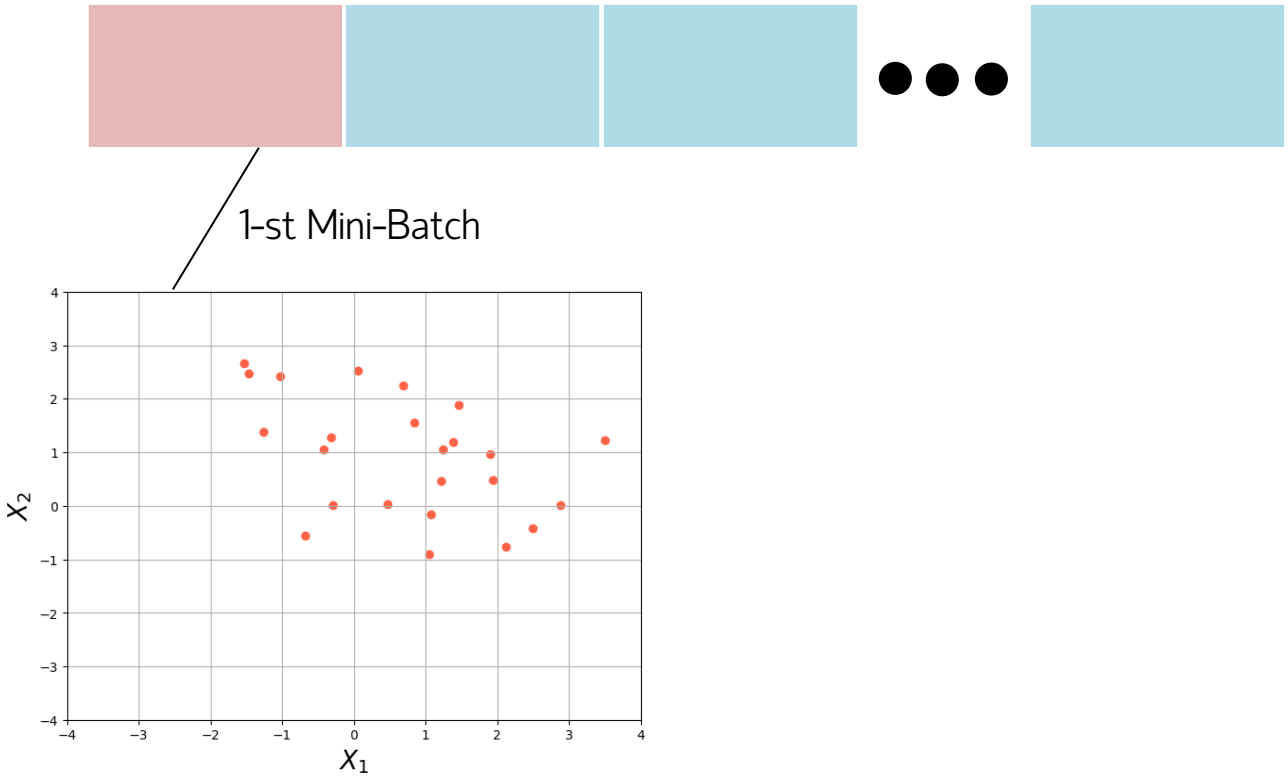


Stochastic Gradient Descent (1 of 7)



In stochastic gradient descent (SGD), we shuffle the training set and break it into mini-batches of B data points; B is simply referred to as the batch size. $M = \lceil N/B \rceil$ is the number of batches.

Stochastic Gradient Descent (2 of 7)



The first mini-batch $\mathcal{B}^{(1)}$ is simply the first B training data points drawn after the data are shuffled.

For m = 1, ..., M (= $\lceil N/B \rceil$)

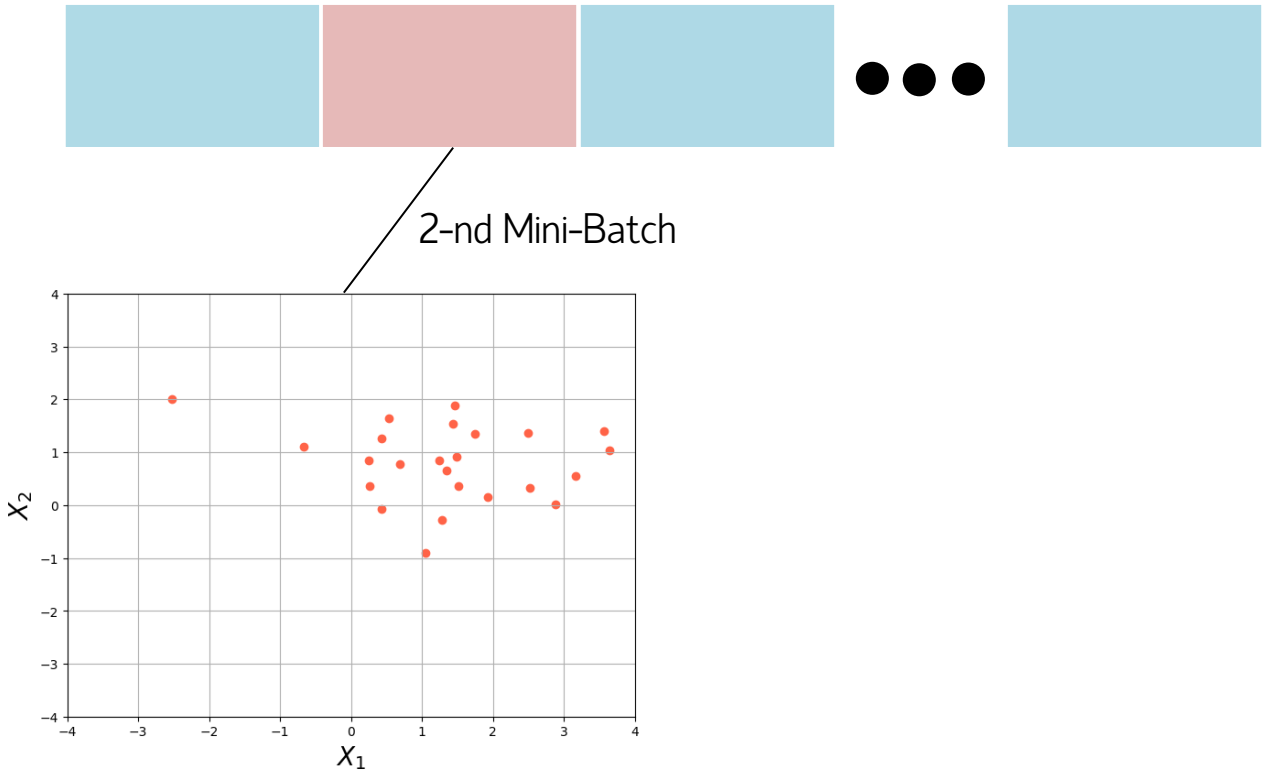
$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$t \leftarrow t + 1$$

The gradients are computed from the data points in the first mini-batch $\mathcal{B}^{(1)}$ $(m = 1)$.

Stochastic Gradient Descent (3 of 7)



The second mini-batch $\mathcal{B}^{(2)}$ contains a fresh, non-overlapping set of training points, completely separate from those in the first mini-batch $\mathcal{B}^{(1)}$.

For m = 1, ..., M (= $\lceil N/_B \rceil$)

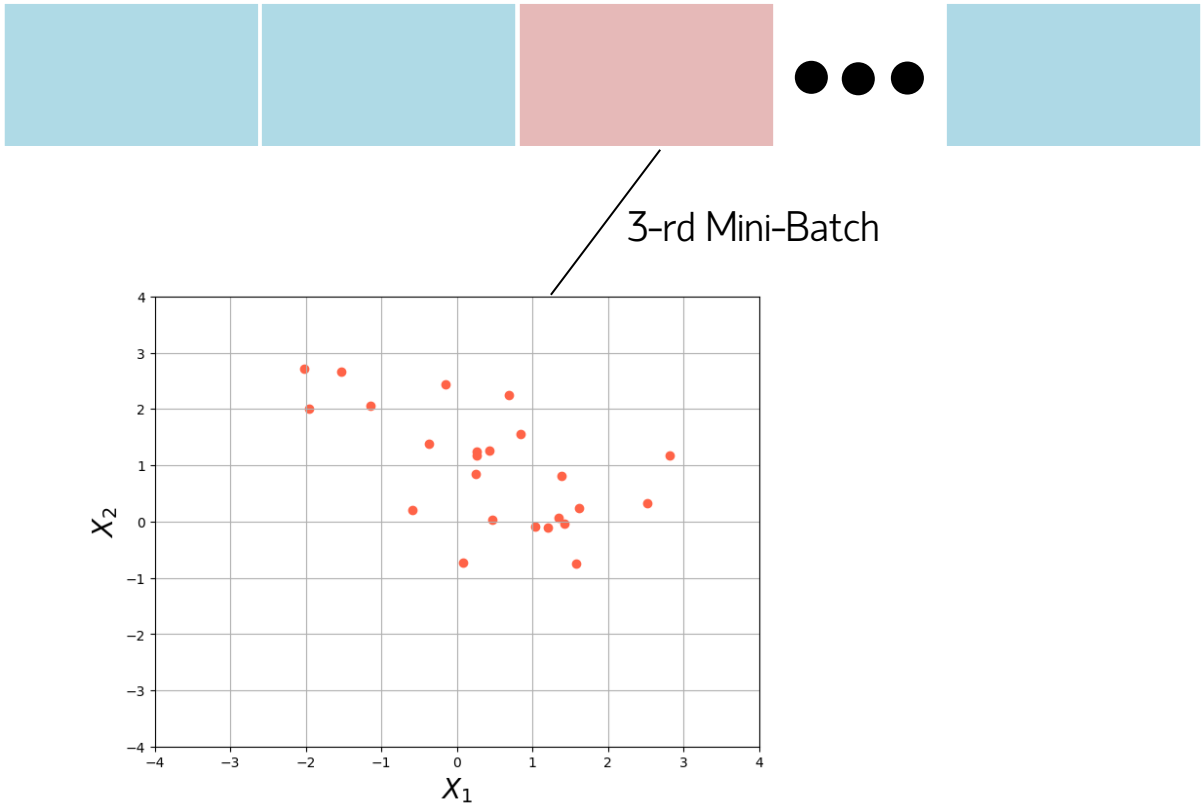
$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$t \leftarrow t + 1$$

The gradients are computed from the data points in the first mini-batch $\mathcal{B}^{(2)}$ $(m = 2)$.

Stochastic Gradient Descent (4 of 7)



The third mini-batch $\mathcal{B}^{(3)}$ contains a fresh, non-overlapping set of training points, completely separate from those in the first $\mathcal{B}^{(1)}$ and second $\mathcal{B}^{(2)}$ mini-batches.

For m = 1, ..., M ($= \lceil N/B \rceil$)

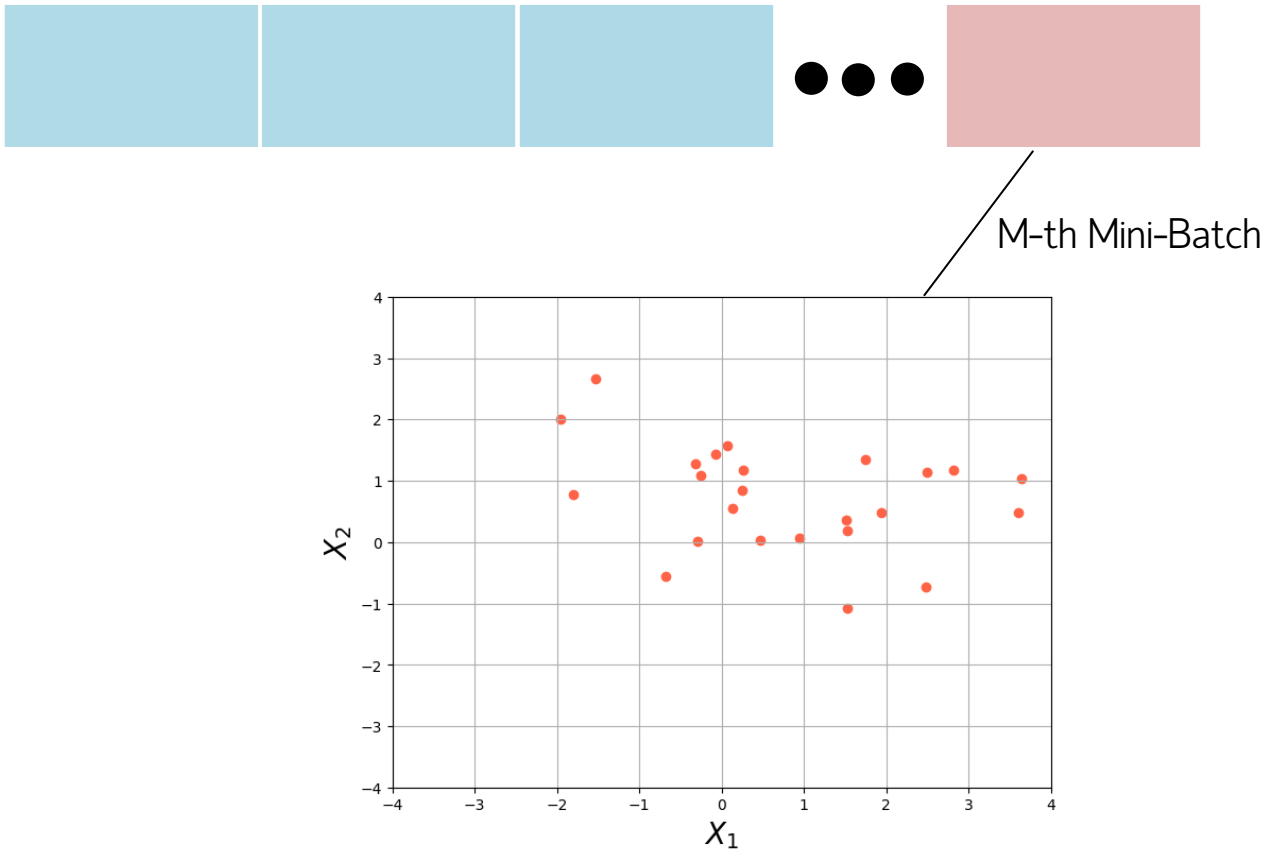
$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$t \leftarrow t + 1$$

The gradients are computed from the data points in the first mini-batch $\mathcal{B}^{(3)}$ $(m = 3)$.

Stochastic Gradient Descent (5 of 7)



For m = 1, ..., M (= $\lceil N/B \rceil$)

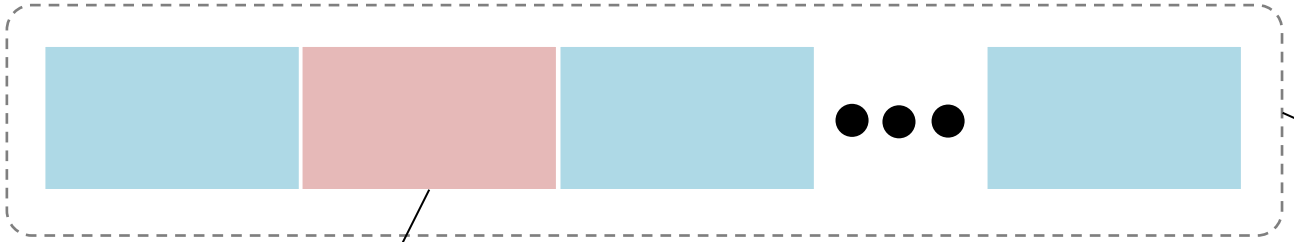
$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

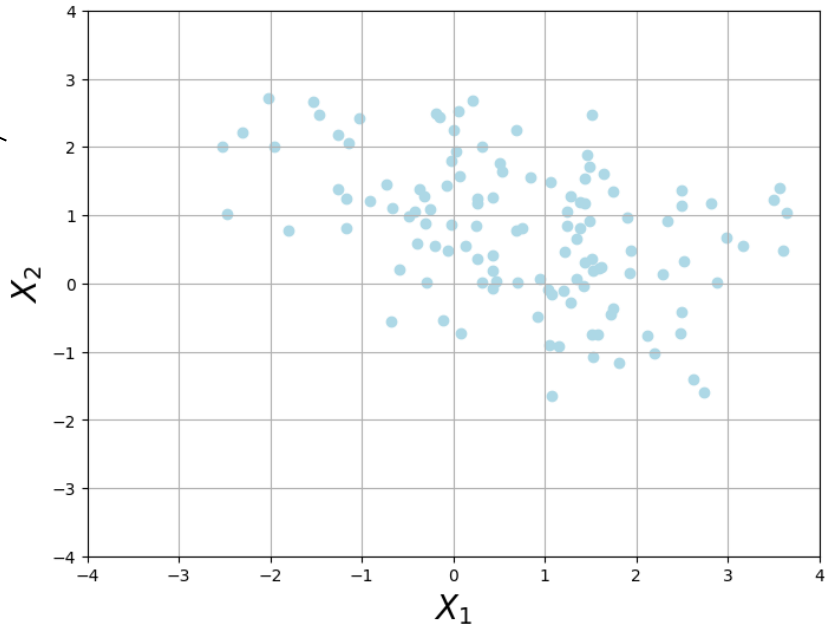
$$t \leftarrow t + 1$$

When we reach $m = M$, every mini-batch has been processed once, all training samples have been seen, and we count that pass through the data as one full epoch

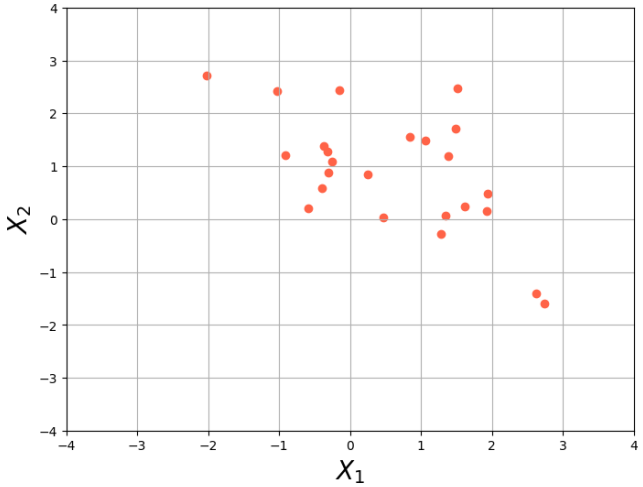
Stochastic Gradient Descent (6 of 7)



Training Data



Mini-Batch

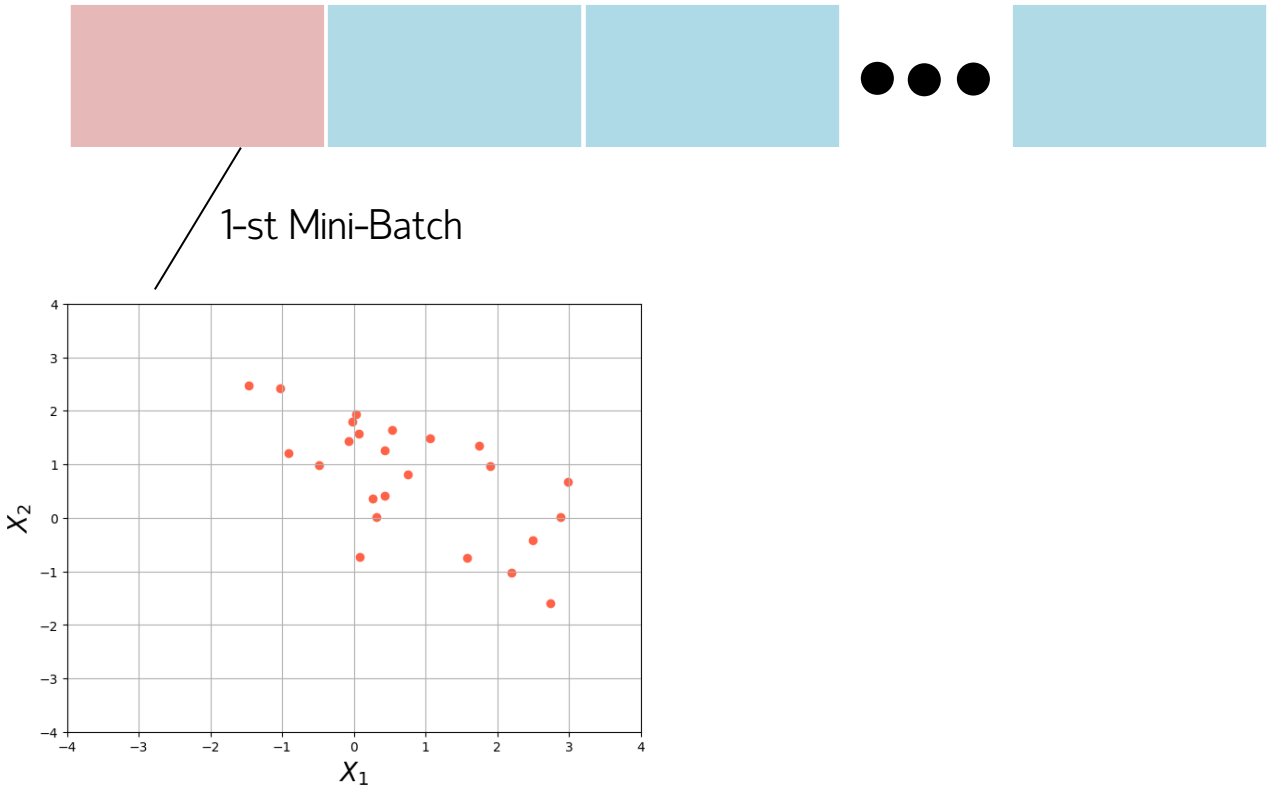


At the start of each new epoch we shuffle the training data again and slice it into mini-batches of size B. As a result, the data points in the second mini-batch of the new epoch differ from those in the second mini-batch of the previous epoch.

We re-shuffle at each epoch to ensure:

- Every example appears in a different mini-batch order each epoch.
- Gradient estimates remain unbiased, with no systematic pattern.
- The model sees a fresh mix of data on every update, improving convergence and generalisation.

Stochastic Gradient Descent (7 of 7)



For m = 1, ..., M (= $\lceil N/B \rceil$)

$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$t \leftarrow t + 1$$

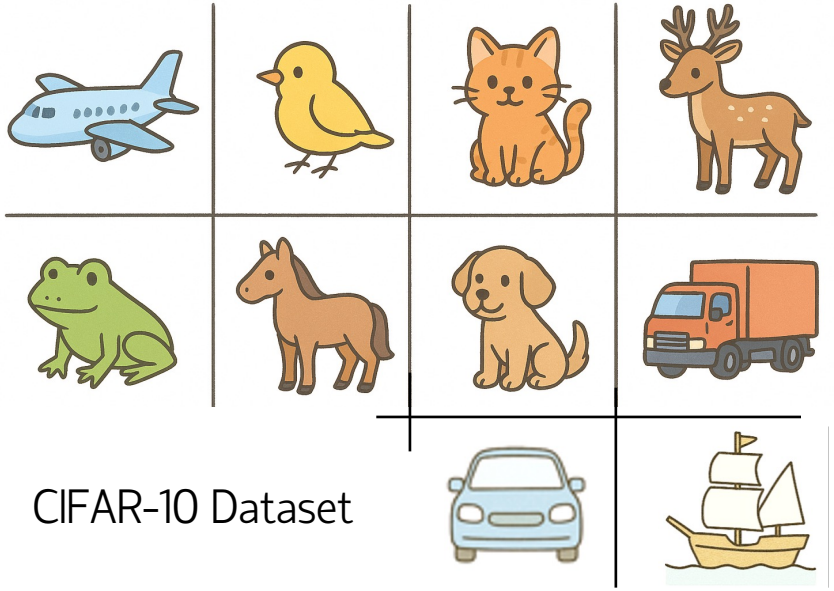
At the start of the new epoch, we draw the first mini-batch $\mathcal{B}^{(1)}$, run a gradient update on it, and then repeat the same process for each subsequent mini-batch until the whole training set has been covered again.

Pseudocode for Stochastic Gradient Descent

```
# Inputs
#   data      ← training set of N examples (x, y)
#   B         ← mini-batch size
#   n         ← learning-rate
#   epochs    ← number of full passes over the data
#   W, b      ← model weights and biases          (initialised)

FOR epoch = 1 TO epochs DO
    shuffle(data)                                # random order each epoch

    split data into consecutive mini-batches ℬ¹, ℬ², …, ℬᴹ
        where M = ceil(N / B)

    FOR each mini-batch ℬᵐ DO
        # ----- forward pass -----
        ŷ ← model(ℬᵐ.x ; W, b)                 # predictions for this batch
        L ← loss(ŷ, ℬᵐ.y)                      # average loss over the batch

        # ----- backward pass -----
        g_W, g_b ← ∇_{W,b} L                   # gradients w.r.t. W and b

        # ----- parameter update -----
        W ← W - n(epoch, m) · g_W
        b ← b - n(epoch, m) · g_b
    END FOR
END FOR
```

Batches: Calculation Example

Component	Details
Input (X)	32 × 32 RGB Image → Tensor Shape 32 × 32 × 3 (3,072 float / uint8 Values)
Label (y)	One of 10 Classes: 0 Airplane · 1 Automobile · 2 Bird · 3 Cat · 4 Deer · 5 Dog · 6 Frog · 7 Horse · 8 Ship · 9 Truck
Training set	50,000 Image–Label Pairs
Test set	10,000 Image–Label Pairs



CIFAR-10 Dataset

Let's reserve 10 k images for validation, use the remaining 40 k for training, and set the batch size to 128.

Q: How many batches will we have?

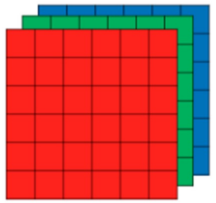
A: $M = \lceil N/B \rceil$. *40k/128 = 312.5 → 313.*

Q: How many data points are there in the 313-rd mini-batch?

A: 40k - 312x128 = 64.

CIFAR-10 Pre-processing: Pixels → Flatten → Normalise

1



3-channel
Image Data

3x[32,32]

2

$$\begin{bmatrix} 255 \\ 231 \\ 42 \\ \vdots \\ 92 \\ 142 \end{bmatrix}$$

Flatten Data

3x32x32 = *3072*

3

$$\begin{bmatrix} 1.0 \\ 0.906 \\ 0.168 \\ \vdots \\ 0.361 \\ 0.557 \end{bmatrix}$$

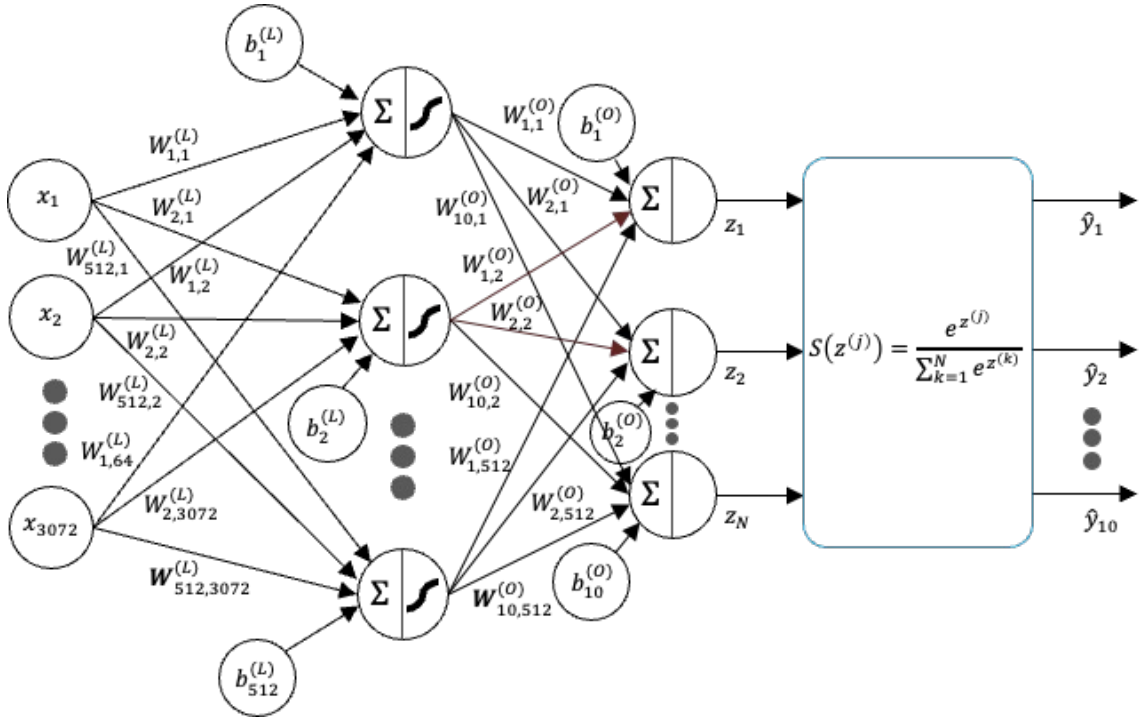
Normalised Data

3072

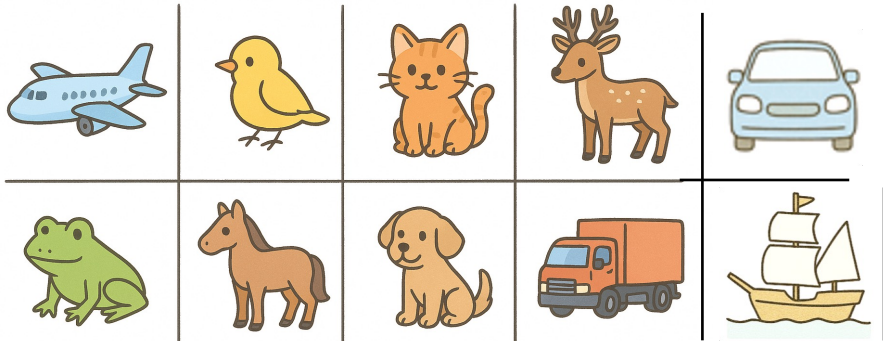
```
from tensorflow.keras import datasets, utils
import numpy as np

# Load CIFAR-10 (50 k train / 10 k test)
(x_train_full, y_train_full), (x_test, y_test) = datasets.cifar10.load_data()

# Flatten images → 32*32*3 = 3072 features
x_train_full = x_train_full.reshape(len(x_train_full), -1)
x_test = x_test .reshape(len(x_test), -1)

# Normalise pixels to [0,1]
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test .astype("float32") / 255.0

# One-hot encode labels
num_classes = 10
y_train_full = utils.to_categorical(y_train_full, num_classes)
y_test = utils.to_categorical(y_test, num_classes)

# Train / validation split (40 k / 10 k)
x_train, x_val = x_train_full[:40_000], x_train_full[40_000:]
y_train, y_val = y_train_full[:40_000], y_train_full[40_000:]
```

TensorFlow

Tensorflow Code Snippet

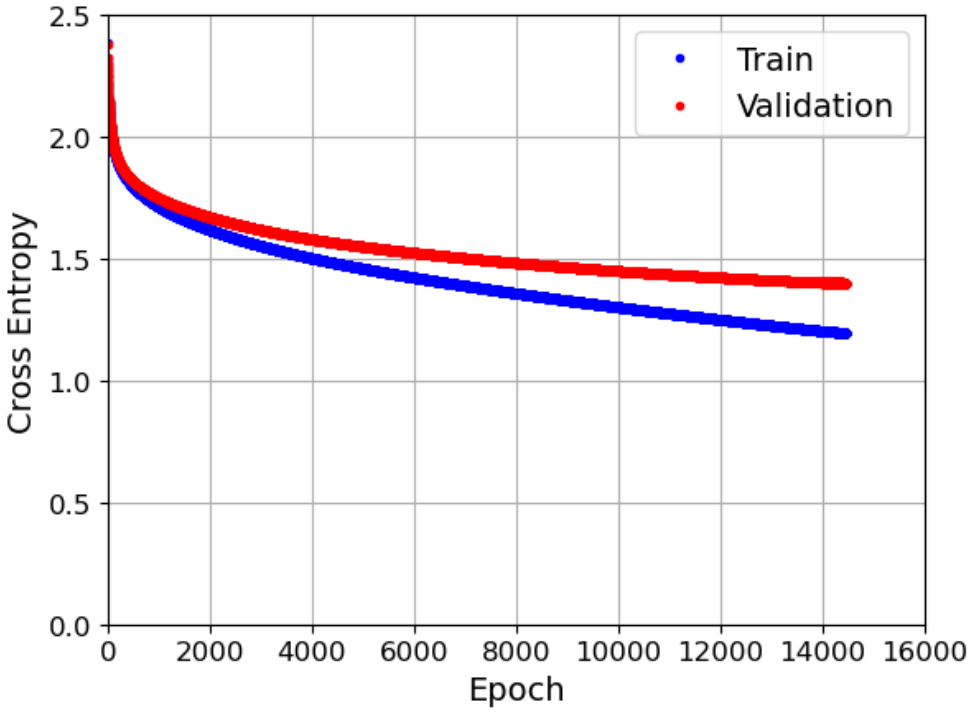


CIFAR-10 Dataset



```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import SGD

num_inputs = 3072          # input dimension
num_hidden_units = 512     # hidden units
num_outputs = 10           # output dimension

model = Sequential([
    Dense(num_hidden_units,
          activation=”relu",
          input_shape=(num_inputs,)),

    Dense(num_outputs,
          activation="softmax”)
])

model.compile(optimizer=SGD(learning_rate=1e-2),
              loss=”categorical_crossentropy",
              metrics=[“accuracy”])

batch_size = 128        # batch size
num_epochs = 100        # number of epochs

early_stop = EarlyStopping(monitor=‘val_loss’,
                           restore_best_weights=True, patience=5)

history = model.fit(X_train, y_train,
                    batch_size=batch_size,
                    epochs=num_epochs,
                    validation_data=(X_val, y_val),
                    callbacks=[early_stop])
```

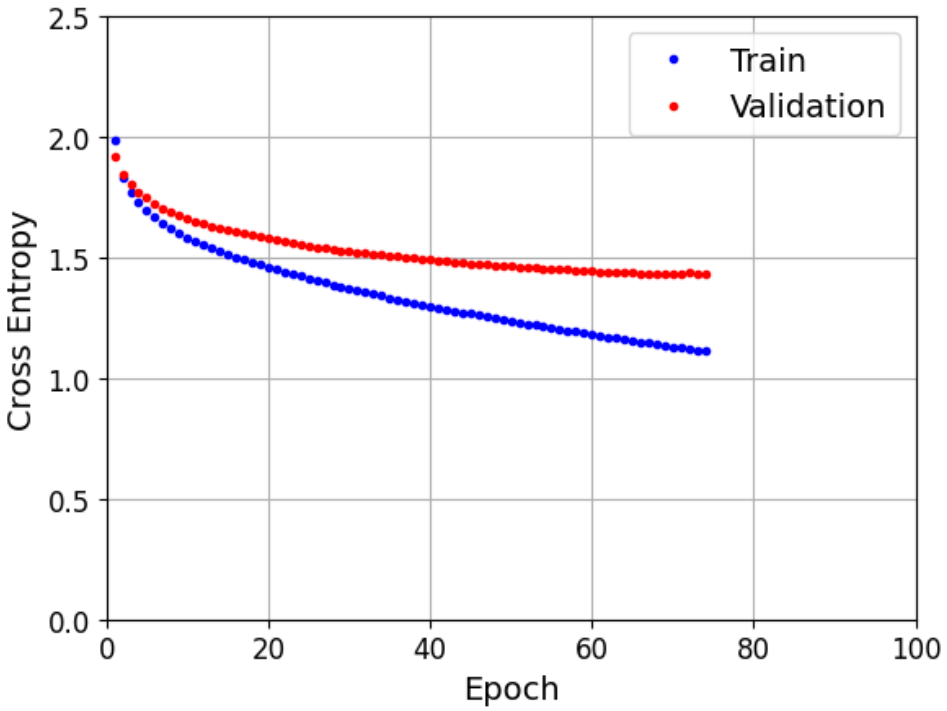
Mini-Batch SGD



- Batch Size: 40,000 (Full Training Data)
- Wall-Clock Time: 14,359 Seconds
- Validation Loss: 1.401
- Validation Accuracy: 0.513

The runs were on Colab with T4 GPU notebook setting.

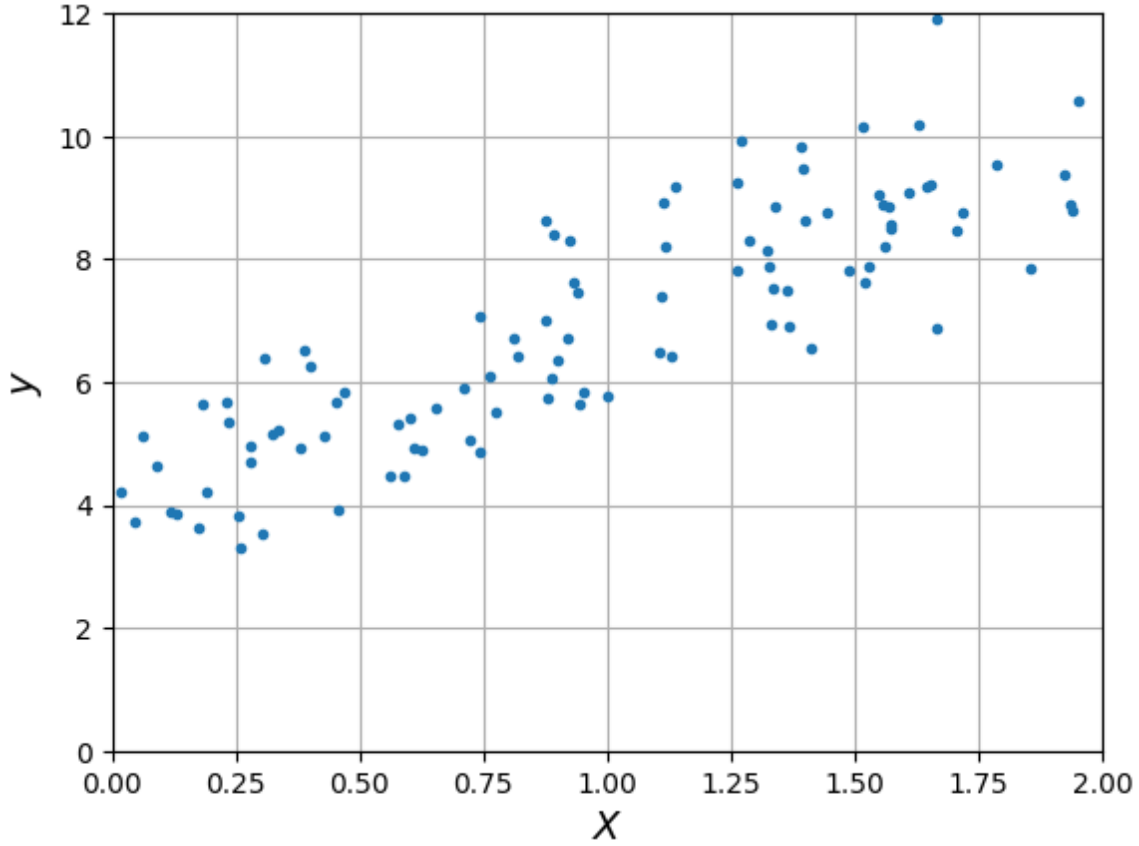
Google Colaboratory



- Batch Size: 128
- Wall-Clock Time 97 Seconds
- Validation Loss: 1.438
- Validation Accurcy: 0.504

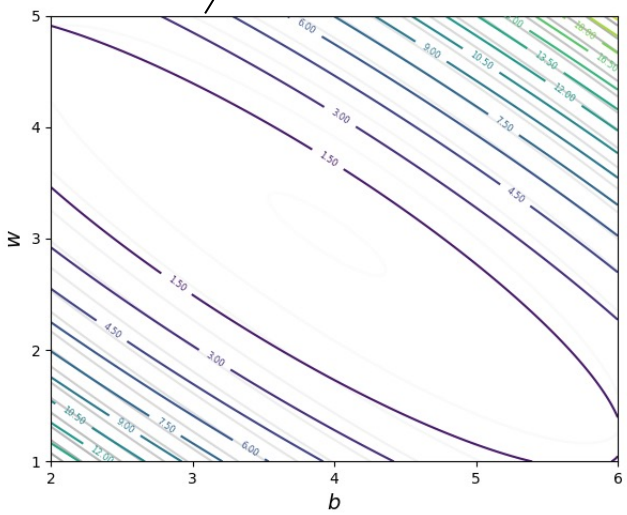
Comparable Accuracy in 1% of the Time

Example Dataset



Mini-Batch Loss and Gradient

1-st Mini-Batch



The loss surface is computed only from the 24 data points in $\mathcal{B}^{(1)}$ $(m = 1)$.

For m = 1, ..., M $(= \lceil N/_B \rceil)$

$$W_{i,j}^{(t+1)} \leftarrow W_{i,j}^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial W_{i,j}} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

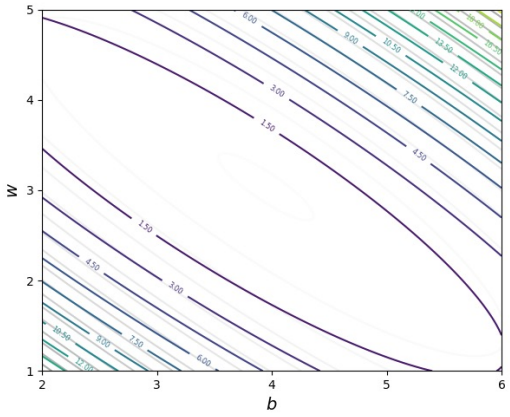
$$b_i^{(t+1)} \leftarrow b_i^{(t)} - \eta \times \left(\sum_{k=1}^{B} \frac{\partial L}{\partial b_i} (W^{(t)}, \vec{b}^{(t)}, \vec{x}^{(m)(k)}) \right)$$

$$t \leftarrow t + 1$$

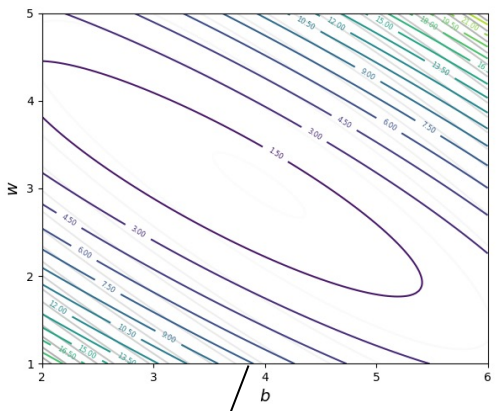
Each iteration uses just one mini-batch to compute both the loss surface and its gradient; the process repeats batch-by-batch until the epoch completes.

Loss Contours

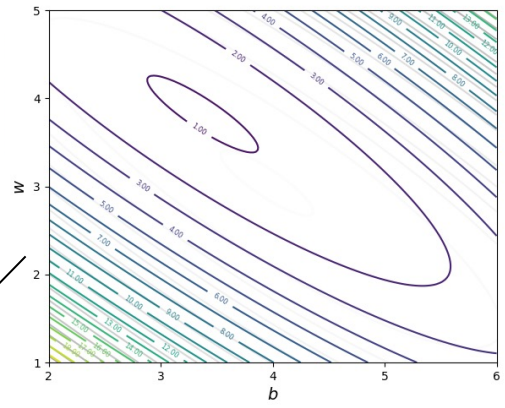
1st Mini-Batch Loss Surface

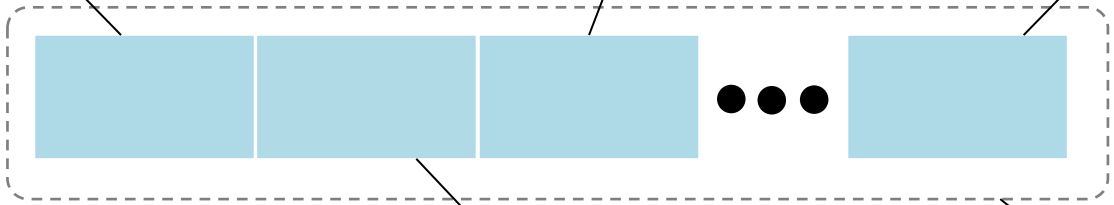


3rd Mini-Batch Loss Surface

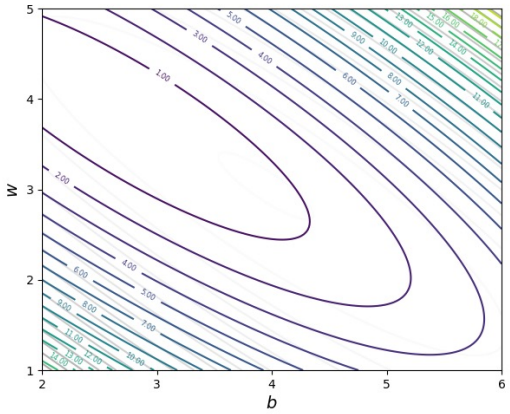


Mth Mini-Batch Loss Surface

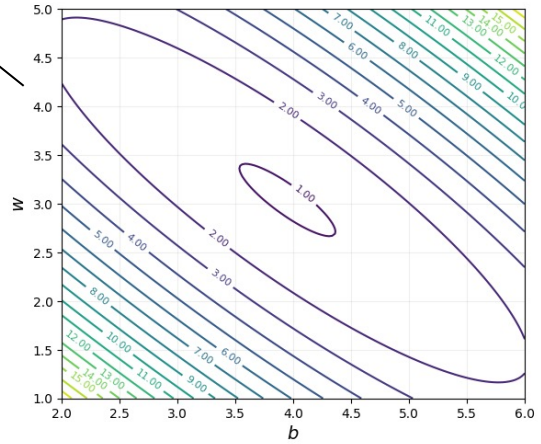




2nd Mini-Batch Loss Surface

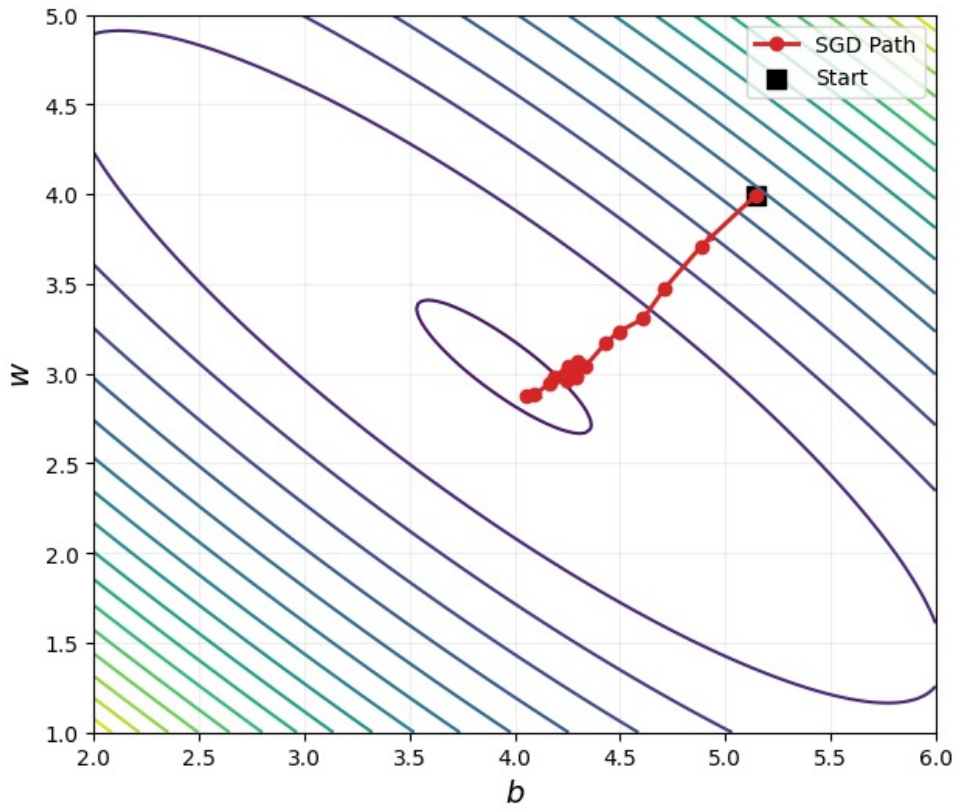


Full Training Loss

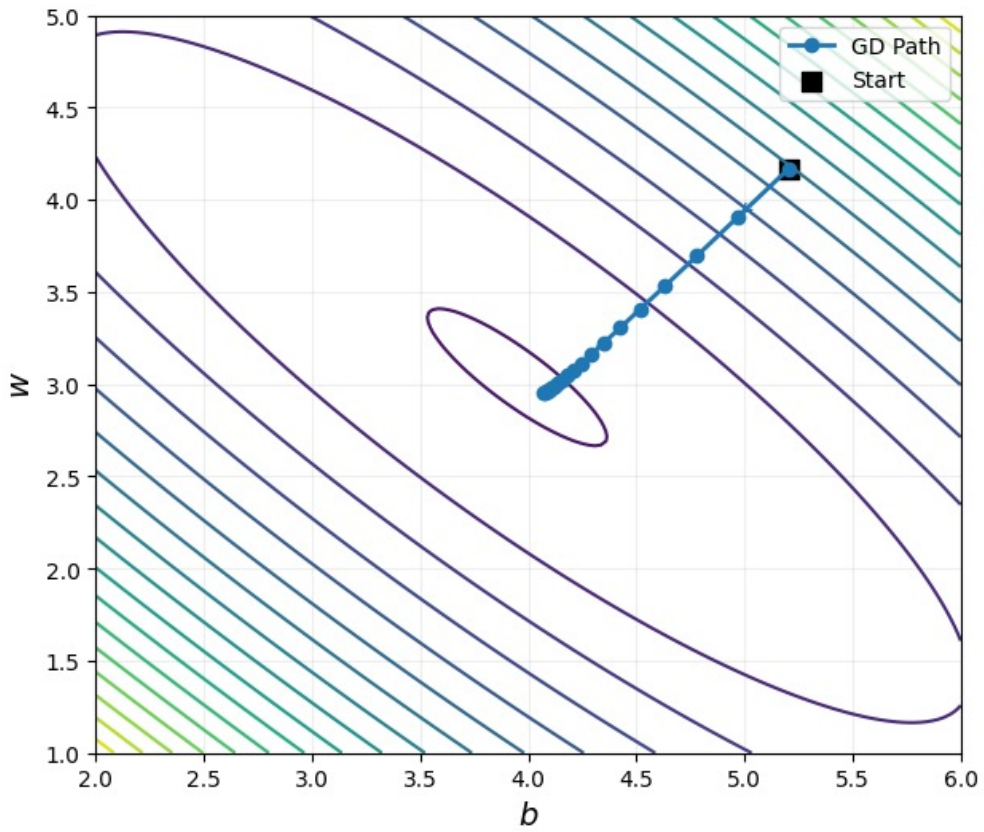


Each batch contains fraction of the data, so its contours wobble around the full-data ellipse, yet the overall shape still points toward the global minimum.

Weight-Update Trajectories: SGD vs Full-Batch Gradient Descent

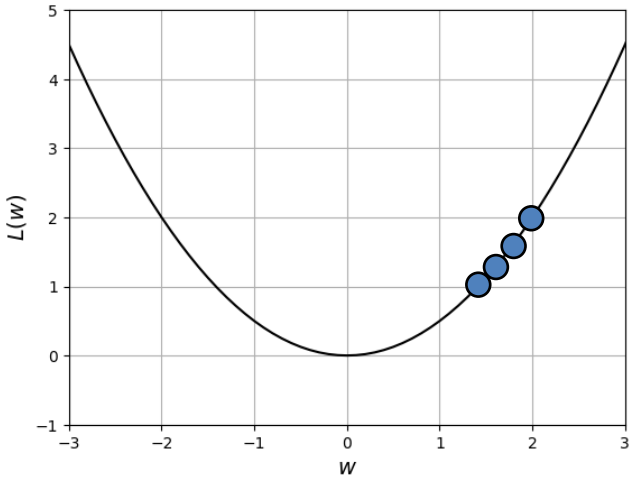


SGD follows the "shifting loss surface". By stepping through one mini-batch after another, SGD explores a series of approximate loss landscapes—cheap to compute, yet accurate enough to guide the weights toward the true minimum.



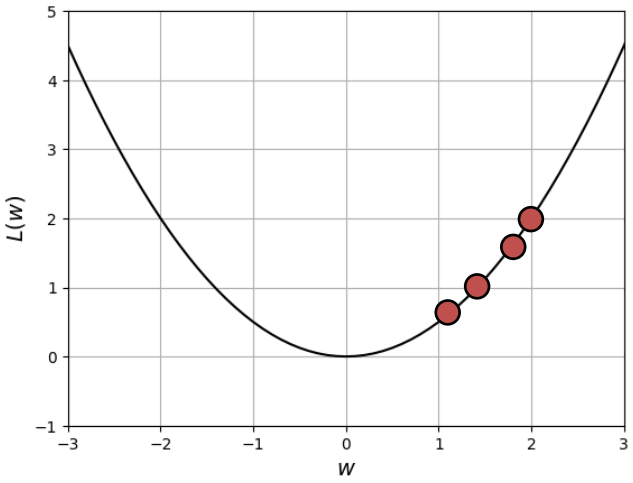
Full-batch GD follows a smooth, direct descent. Because every update uses the *exact* gradient of the whole training set, the path glides along a single, unchanging loss surface. There is no zig-zagging or back-tracking caused by mini-batch noise.

SGD with Momentum



Gradient Descent Algorithm:

$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{\partial L}{\partial w}\left(w^{(t)}\right)$$



Momentum Algorithm:

$$v^{(t+1)} \leftarrow \beta v^{(t)} - \eta \frac{\partial L}{\partial w}\left(w^{(t)}\right)$$

$$w^{(t+1)} \leftarrow w^{(t)} + v^{(t)}$$

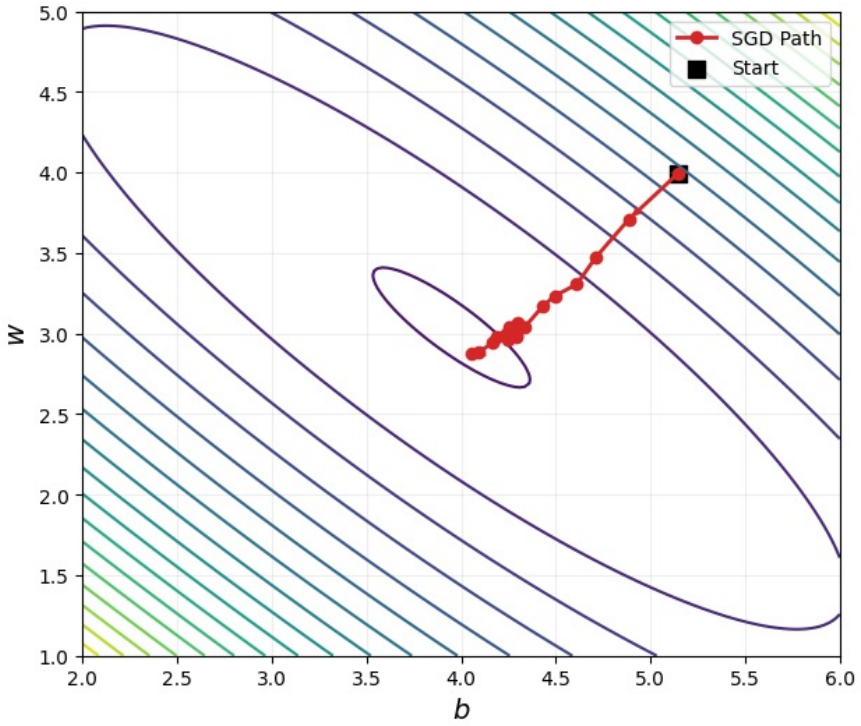
Iteration	SGD Weight $\left(w^{(t)}\right)$	SGD Step $\left(\eta \frac{\partial L}{\partial w}\left(w^{(t)}\right)\right)$
0	2.000	–
1	1.800	0.200
2	1.620	0.180
3	1.458	0.162

Iteration	SGDM Weight $\left(w^{(t)}\right)$	SGDM Gradient $\left(\eta \frac{\partial L}{\partial w}\left(w^{(t)}\right)\right)$	SGDM Step $\left(v^{(t)}\right)$
0	2.000	–	–
1	1.800	0.200	0.200
2	1.440	0.180	0.360
3	0.972	0.144	0.468

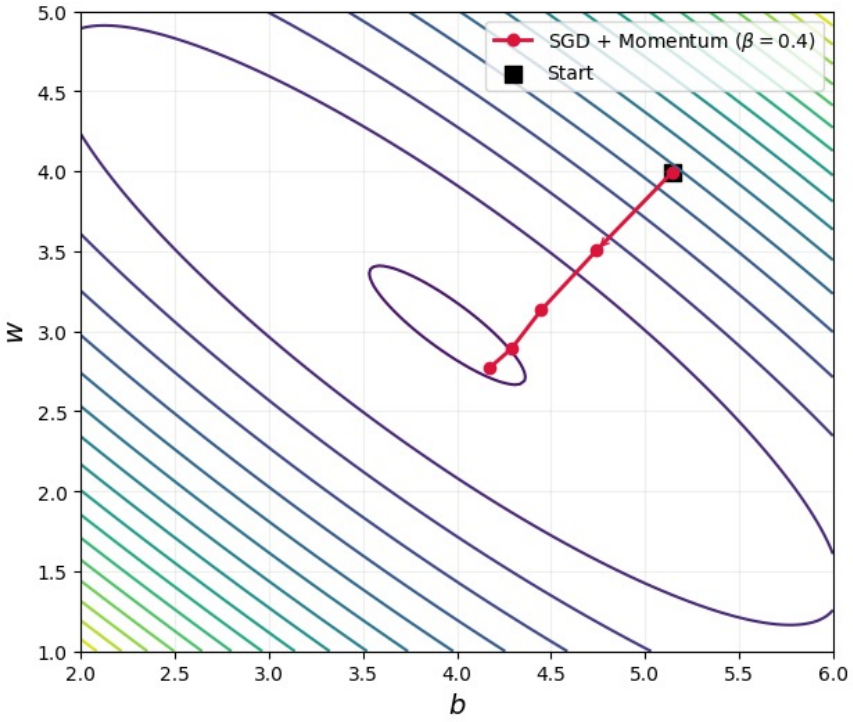
Setup: $\eta = 0.1,\ \beta = 0.9,\ w^{(0)} = 2.0$

- When successive gradients point the same way, $v^{(t)}$ compounds: steps grow (0.200 → 0.468) and the optimiser "builds speed."
- As the slope flattens or flips sign, the new gradient opposes the stored $v^{(t)}$. Velocity shrinks, so steps shorten *without* manual LR tweaks.

Batch-Noise Cancellation via Momentum



+ Momentum $\left(\beta v^{(t)}\right)$



Each batch's gradient points a slightly different way, so the weight vector keeps oscillating across the valley walls.

Accumulated velocity keeps the update pointing in a consistent direction → longer, straighter trajectories toward the minimum.

*Momentum treats gradients like a rolling average: the $\beta v^{(t)}$ term **amplifies correlated gradients and damps directional conflicts, yielding a smoother optimization trajectory.***

Optimisers: Speed and Stability

Optimizer	Small / Shallow Datasets	Large / Deep Models
SGD	Starts slowly and needs a well-chosen learning-rate schedule to avoid stalling. Once tuned, it learns clean, often flatter solutions that generalise well.	Scales to big data with modest memory, but still lags others in early epochs; step-decay or cosine LR is almost mandatory. Stable when tuned, but high LR can explode training.
SGD + Momentum	Momentum (≈0.9) accelerates progress and damps zig-zags, so convergence is noticeably faster than vanilla SGD. Usually reaches the best test accuracy on small sets after a few LR drops.	The workhorse of vision: with cosine or step decay it approaches adaptive optimisers' speed but keeps SGD's strong generalisation. Overshoot is rare thanks to momentum's damping.
Adagrad	Per-feature step-size boosts rare features and gives lightning-fast first epochs, especially on sparse UCI data. Its ever-shrinking LR then causes an early plateau in accuracy.	Same rapid early drop on deep nets, but learning stalls when LR decays to near-zero; final accuracy often trails others. Safe and rarely diverges, yet hard to "un-freeze" later.
RMSProp	Maintains a running RMS of past gradients, so it adapts quickly without the aggressive decay that hobbles Adagrad. Stable on tricky RNN toy problems with minimal tuning.	Matches Adam's early loss reduction on CNNs and small transformers, and usually trains without oscillation. May need LR decay + ε tuning to squeeze out the very last percent.
Adam	Combines RMSProp + momentum, giving the fastest loss drop and tolerance to higher LR; perfect for quick prototyping. Can overfit tiny datasets unless you add weight-decay or early-stop.	Default choice for transformers: reaches target accuracy in the fewest steps and scales to huge batches. Extremely stable across wide LR ranges, though it may land in slightly sharper minima unless weight-decay (AdamW) is used.

Summary

- Mini-batch SGD gives the same destination for a fraction of the cost. By cycling through many small, shuffled batches each epoch, the updates collectively approximate the full-data gradient, so we reach virtually the same model while avoiding one huge, slow pass over the entire dataset.
- Batch-to-batch noise is a built-in regulariser. Because each batch sees only a slice of the data, its loss surface "wobbles" around the true contour; this stochasticity shakes the weights out of sharp minima and keeps training from over-fitting .
- Momentum smooths that noise and accelerates descent. The extra term $\beta v^{(t)}$ forms a rolling average of recent gradients: correlated directions reinforce, conflicting ones cancel. The result is a straighter, longer stride toward the minimum—faster convergence with fewer zig-zags.